

Assignment 6 — SQAT

Student environment: macOS (local runner), Remote execution: BrowserStack

Website under test: Booking.com

Generated: 2026-01-30 22:55:57

Overview

Task 1: Read test data from Excel using openpyxl (no hardcoded test values).

Task 2: Run Selenium tests on BrowserStack using Remote WebDriver on at least 2 browsers and 1 OS.

Evidence included: code snippet, proof of Excel data read, screenshots of execution, and short explanations.

Task 1 — Test data from Excel (openpyxl)

Excel file used: testdata.xlsx

Sheet: data

All input values (URL, destination, date offsets, guests) are read from Excel.

The automation logic does not contain hardcoded test values.

key	value
base_url	https://www.booking.com/
destination	Astana
checkin_offset_days	14
checkout_offset_days	17
adults	2
rooms	1
take_screenshots	yes
wait_timeout_sec	25

Figure 1 — Proof image: data read from Excel and used by the test

Task 2 — BrowserStack execution (Selenium Remote WebDriver)

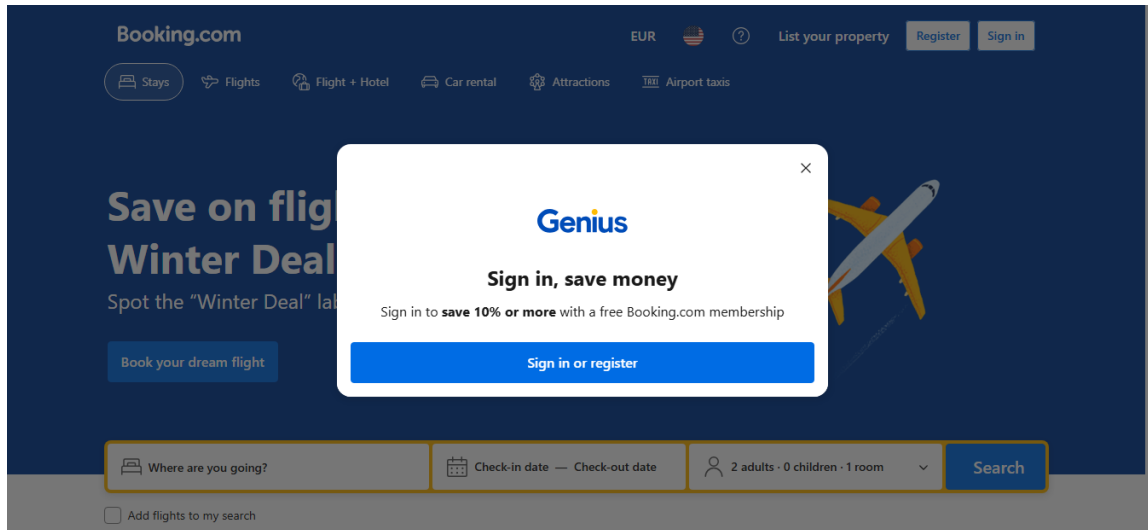
Chosen OS: Windows 11

Browsers: Chrome (latest), Firefox (latest)

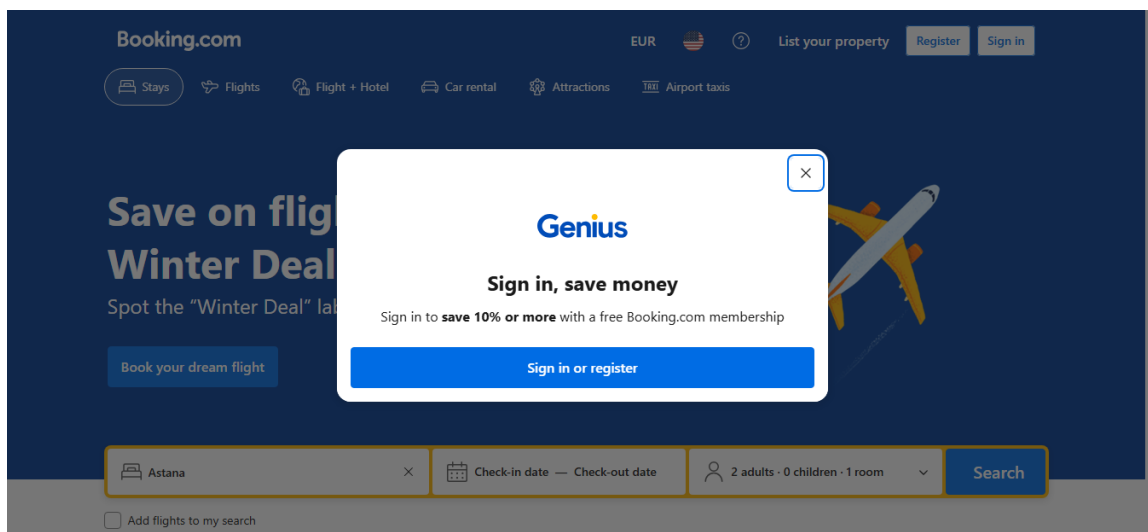
Rationale: running on two different browsers helps detect browser-specific UI/DOM differences.

Screenshots are captured at key steps: home page, destination filled, dates selected, guests set (if available), results page.

Run: Windows 11 — Chrome (latest) — PASSED=False



Windows_11_Chrome: Home page opened



Windows_11_Chrome: Error screenshot (if failed)

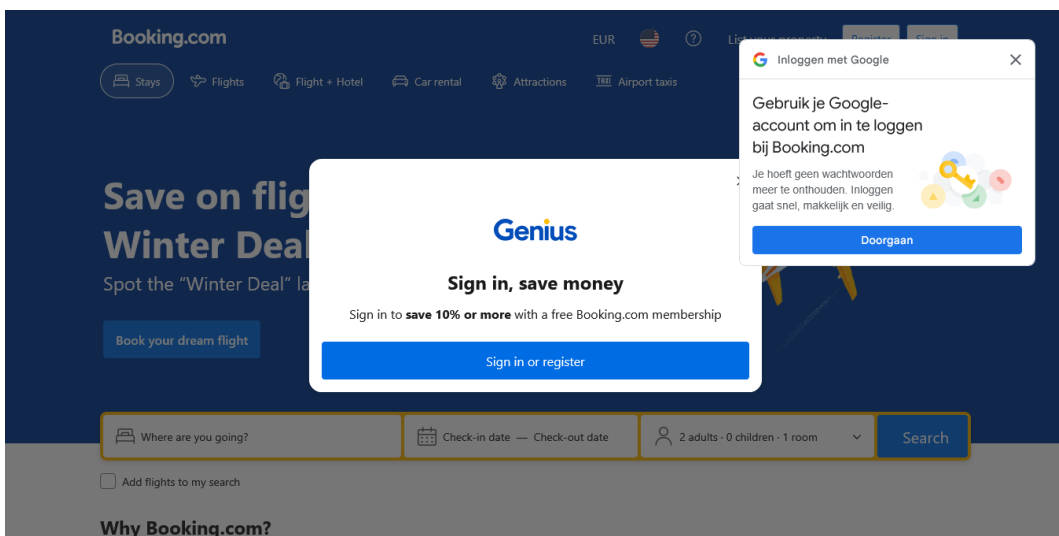
Status: FAILED: TimeoutException: Message:

Stacktrace:

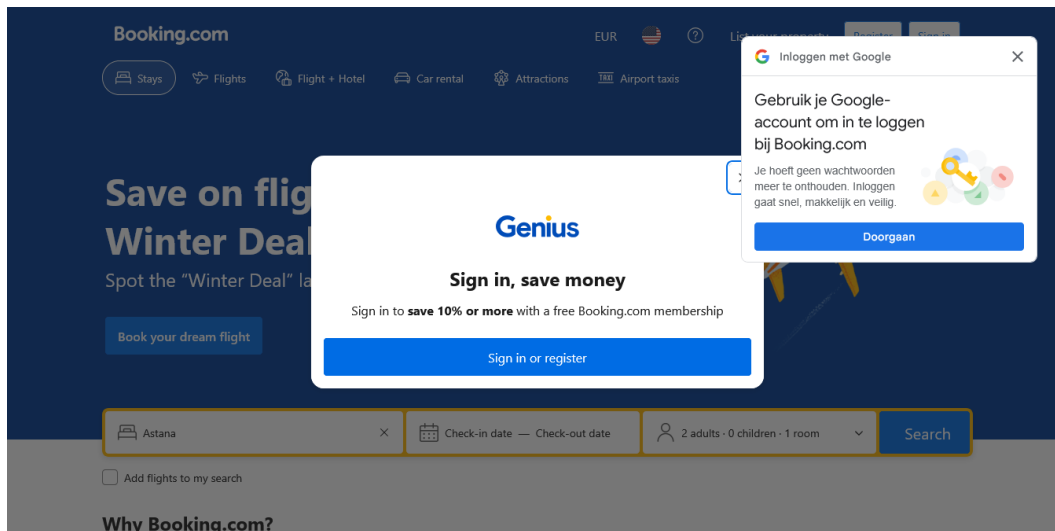
Symbols not available. Dumping unresolved backtrace:

```
0x7ff66a9ba545
0x7ff66a9ba5a0
0x7ff66a760fed
0x7ff66a7bc6d2
0x7ff66a7bc9dc
0x7ff66a80d5b7
0x7ff66a80a10c
0x7ff66a7acedc
0x7ff66a7ade23
0x7ff66ac98bb0
0x7ff66ac92fbd
0x7ff66acb420a
0x7ff66a9d6e15
0x7ff66a9df4dc
0x7ff66a9c3e54
0x7ff66a9c4006
0x7ff66a9a9e27
0x7ff8b4ffe8d7
0x7ff8b5c08d9c
```

Run: Windows 11 — Firefox (latest) — PASSED=False



Windows_11_Firefox: Home page opened



Windows_11_Firefox: Error screenshot (if failed)

Status: FAILED: TimeoutException: Message:

Stacktrace:

RemoteError@chrome://remote/content/shared/RemoteError.sys.mjs:8:8

WebDriverError@chrome://remote/content/shared/webdriver/Errors.sys.mjs:202:5

NoSuchElementException@chrome://remote/content/shared/webdriver/Errors.sys.mjs:555:

5

dom.find/</<@chrome://remote/content/shared/DOM.sys.mjs:136:16

Code snippet (evidence)

```
import os
import sys
import time
import json
import textwrap
import traceback
from datetime import datetime
from pathlib import Path

from openpyxl import Workbook, load_workbook

from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.common.keys import Keys
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.chrome.options import Options as ChromeOptions
from selenium.webdriver.firefox.options import Options as FirefoxOptions

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

from reportlab.lib.pagesizes import A4
from reportlab.lib.units import mm
from reportlab.lib.utils import ImageReader
from reportlab.pdfgen import canvas

# =====
# CONFIG / CONSTANTS
# =====
PROJECT_NAME = "Assignment 6 — SQAT"
TEST_SITE_NAME = "Booking.com"
DEFAULT_EXCEL = "testdata.xlsx"
DEFAULT_SHEET = "data"
```

```

ws["A1"] = "key"
ws["B1"] = "value"

# IMPORTANT: NO HARDCODED values in test logic; they come from Excel.
# You may edit these values later.
rows = [
    ("base_url", "https://www.booking.com/"),
    ("destination", "Astana"),
    ("checkin_offset_days", 14), # check-in is today + offset
    ("checkout_offset_days", 17), # check-out is today + offset
    ("adults", 2),
    ("rooms", 1),
    ("take_screenshots", "yes"),
    ("wait_timeout_sec", 25),
]

for i, (k, v) in enumerate(rows, start=2):
    ws[f"A{i}"] = k
    ws[f"B{i}"] = v

wb.save(excel_path)

def read_test_data(excel_path: Path, sheet_name: str) -> dict:
    wb = load_workbook(excel_path)
    if sheet_name not in wb.sheetnames:
        raise ValueError(f"Sheet '{sheet_name}' not found. Available: {wb.sheetnames}")
    ws = wb[sheet_name]

    data = {}
    for row in ws.iter_rows(min_row=2, values_only=True):
        if not row or row[0] is None:
            continue
        key = str(row[0]).strip()
        value = row[1]
        data[key] = value

    # minimal validation
    required = ["base_url", "destination", "checkin_offset_days", "checkout_offset_days", "adults", "rooms"]
    missing = [k for k in required if k not in data]
    if missing:
        raise ValueError(f"Missing required keys in Excel: {missing}")

    return data

def make_data_proof_image(data: dict, out_png: Path) -> None:
    """
    Creates a PNG table image showing the Excel data that was read.
    This is convenient proof for the PDF report.
    """
    keys = list(data.keys())
    vals = [str(data[k]) for k in keys]

    fig = plt.figure(figsize=(9, max(3, 0.35 * len(keys) + 1)))
    ax = fig.add_subplot(111)
    ax.axis("off")

    table_data = [["key", "value"]] + [[k, v] for k, v in zip(keys, vals)]
    tbl = ax.table(cellText=table_data, loc="center")

    # Keep default matplotlib styling (no custom colors).
    tbl.auto_set_font_size(False)
    tbl.set_fontsize(10)
    tbl.scale(1, 1.3)

    fig.tight_layout()
    fig.savefig(out_png, dpi=200)
    plt.close(fig)

# =====
# TASK 2: BROWSERSTACK TEST

```

```
run_log = {
    "base_url": base_url,
    "destination": destination,
    "checkin_offset_days": checkin_offset,
    "checkout_offset_days": checkout_offset,
    "steps": [],
    "passed": False,
    "error": None,
}

def step(name: str):
    run_log["steps"].append({"time": datetime.now().isoformat(timespec="seconds"), "name": name})

# Step 1: Open site
step("Open Booking.com")
driver.get(base_url)
time.sleep(2)
accept_cookies_if_present(driver, wait)
if take_screenshots:
    safe_screenshot(driver, artifacts_dir / "01_home.png")

# Step 2: Fill destination
step("Fill destination")
# Booking.com uses name="ss" often
dest_input = wait.until(EC.element_to_be_clickable((By.NAME, "ss")))
```